

COLLEGE MINI-PROJECT • CLI-BASED PYTHON APPLICATION

Lost & Found Item Tracking System

A lightweight command-line tool for reporting, searching, and recovering lost or found items on campus.

Team Members: Lohith • Prajwal • Manjegowda • Nithyashree • Manu • Prashant

SPEAKER NOTES

Welcome the audience and introduce the project in one line: "a simple CLI tool that lets students post, search, and resolve lost and found items without needing a server or database." Introduce team members briefly by name. Mention this is a mini-project focused on core Python, file-based storage, and modular design.

Problem Statement

- ▶ Lost items on campus — ID cards, bottles, electronics, bags — are hard to track once misplaced
- ▶ Current methods rely on noticeboards, word-of-mouth, or informal social media groups
- ▶ There is no central record, so genuinely found items often go unclaimed or unreported
- ▶ Owners and finders have no structured, direct way to connect with each other
- ▶ Need: a simple system anyone can run instantly — no internet, server, or database setup required

SPEAKER NOTES

Open with a relatable scenario — e.g. losing a water bottle or ID card in the library with no clear way to report it. Use this to justify why a lightweight CLI tool is a practical first step, rather than jumping straight to a full web app.

Objectives

- ▶ Build a command-line system to report lost and found items
- ▶ Allow users to attach images to items for easier identification
- ▶ Provide search and filtering by keyword, category, or location
- ▶ Let users update an item's status — Lost, Found, or Resolved
- ▶ Implement basic user accounts (register / login) for accountability
- ▶ Enable direct contact between the finder and the owner
- ▶ Keep the system lightweight — no external database or server dependency

SPEAKER NOTES

Frame these as "what we set out to build." Each objective here maps directly to a feature covered on the Features slide — mention that connection so the audience sees the through-line from goal to implementation.

Tech Stack

- ▶ **Language:** Python 3 (standard library only)
- ▶ **Data Storage:** JSON files — `items.json`, `users.json`
- ▶ **Image Storage:** local folder — `data/images/`
- ▶ **Interface:** Command-Line Interface (CLI), menu-driven navigation
- ▶ **Dependencies:** none external — no database server, no internet connection needed
- ▶ **Portability:** runs on any machine with Python installed

SPEAKER NOTES

Emphasize that simplicity was a deliberate design choice for a mini-project timeline. JSON keeps data human-readable and easy to debug, and avoids the setup overhead of installing MySQL/PostgreSQL for a small-scale prototype.

System Architecture & Data Flow



- ▶ User interacts only through the CLI menu — no direct file access
- ▶ The menu routes each choice to the matching core function
- ▶ Functions operate on `Item` / `User` objects defined in `models.py`
- ▶ The `Database` class centralizes all JSON reads/writes so data stays consistent
- ▶ Uploaded images are saved in `data/images/` and referenced by filename inside `items.json`

SPEAKER NOTES

Walk through the diagram left to right using one concrete example — e.g. "when a user posts an item, the request flows from the menu into `post_item()`, which creates an `Item` object and hands it to the `Database` class to be saved into `items.json`." Note: this diagram is a description only — the final visual version will be hand-drawn separately.

Features

- ▶ **Post Lost/Found Items** — add name, category, location, date, and description
- ▶ **Upload Images** — attach a photo to an item for visual identification
- ▶ **View All Items** — browse the complete list of reported items
- ▶ **Search Items** — filter by keyword, category, or location
- ▶ **Update Item Status** — mark as Lost, Found, or Resolved
- ▶ **User Registration & Login** — basic authentication for accountability
- ▶ **Contact Owner** — reach out via the email or phone linked to an item

SPEAKER NOTES

If a live demo isn't possible, mention that terminal screenshots for each of these flows are coming up shortly. Good slide to briefly demo one or two features live if time and setup allow.

Code Structure & Module Diagram

```
lost_found_system/  
├─ main.py           # CLI entry point & menu-driven interface  
├─ models.py         # Item, User, Database classes  
├─ functions.py      # post_item(), view_all_items(), search_item(),  
│                     #   update_status(), register(), login(),  
│                     #   contact_owner(), _handle_image_upload()  
├─ data/  
│   ├─ items.json    # all posted item records  
│   ├─ users.json    # registered user records  
│   └─ images/       # uploaded item photos  
└─ README.md         # setup & usage instructions
```

SPEAKER NOTES

Highlight that models.py centralizes the Item/User/Database definitions — this shared foundation is what let six team members build separate features in parallel without conflicting data structures.

Team Contributions

NAME	CONTRIBUTION
Lohith	Project skeleton, <code>models.py</code> (Item / User / Database classes), README
Prajwal	<code>post_item()</code> and <code>view_all_items()</code> functions
Manjegowda	Image upload support (<code>_handle_image_upload</code>)
Nithyashree	Search items and update status functions
Manu	User login, register, and contact owner features
Prashant	Project presentation (PPT)

SPEAKER NOTES

Credit each teammate by name as it's read out. Mention that the shared `models.py` contract (built early by Lohith) is what allowed the rest of the team to build their features independently and merge with minimal conflicts.

Screenshots (Terminal Output)

Main Menu(all options listed)

Post New Item(input prompts + confirmation)

Image Upload(file path prompt + success message)

View All Items(formatted list output)

Search Results(keyword / category / location)

Update Status(before & after state)

Register / Login(success & error cases)

Contact Owner(email / phone displayed)

items.json Snippet(raw data structure, optional)

SPEAKER NOTES

Narrate each screenshot as a mini-story: what the user typed, what the system did, and what output confirmed it worked. If time allows, cover error cases too (e.g. invalid login) to show robustness.

Challenges Faced

- ▶ Keeping the Item / User data structure consistent while 6 people built features in parallel
- ▶ Correctly storing image file paths and keeping them in sync with `items.json`
- ▶ Avoiding JSON read/write conflicts when multiple functions touch the same file
- ▶ Validating user input safely in a CLI with no GUI-level input constraints
- ▶ Designing a menu flow that stays simple and doesn't confuse first-time users
- ▶ Merging code from 6 contributors without breaking functions others depended on

SPEAKER NOTES

Keep this honest and specific rather than generic. Pick two or three points to expand on naturally if asked during Q&A — panels often probe this slide the most.

Future Scope

- ▶ Migrate from JSON files to a proper database (SQLite / PostgreSQL) for scalability
- ▶ Build a web or GUI interface (Flask / Django, or Tkinter) on top of the same core logic
- ▶ Add email or SMS notifications when a matching item is reported
- ▶ Introduce basic image-similarity matching to auto-suggest possible matches
- ▶ Add an admin dashboard to moderate and verify posted items
- ▶ Deploy as a hosted web app for campus-wide access beyond a single machine

SPEAKER NOTES

Present these as natural next steps rather than missing features — it shows the team understands the current system's boundaries and has a realistic upgrade path in mind.

Conclusion

- ▶ Built a working, lightweight CLI system for tracking lost and found campus items
- ▶ Covers the full loop: post → search → update status → contact owner
- ▶ JSON-based storage kept the project simple, transparent, and easy to debug
- ▶ Demonstrates practical use of OOP design (`models.py`), file I/O, and modular Python structure
- ▶ Provides a solid foundation for future upgrades — database, web UI, notifications

SPEAKER NOTES

Close the loop by connecting back to the Objectives slide — confirm each objective was met by the final system. Keep this brief and confident.

Thank You

Questions & Feedback Welcome

Team: Lohith • Prajwal • Manjegowda • Nithyashree • Manu • Prashant

SPEAKER NOTES

Thank the faculty/audience for their time, invite questions, and offer to walk through any specific feature or code file in more depth if asked.